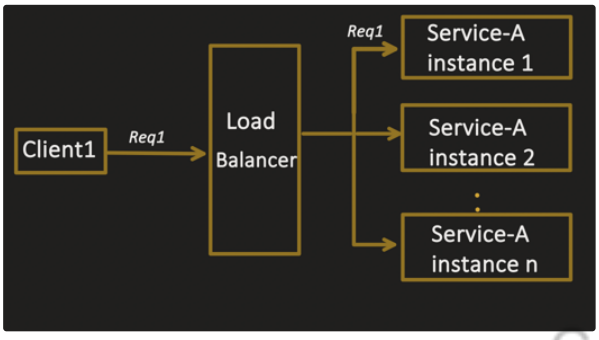
Client Side Load Balancer

Load Balancer:

- Helps in distributing traffic to multiple instances of a servers.
- Helps in preventing single server from being overloaded with huge traffic.

<u>Load Balancer Types:</u>

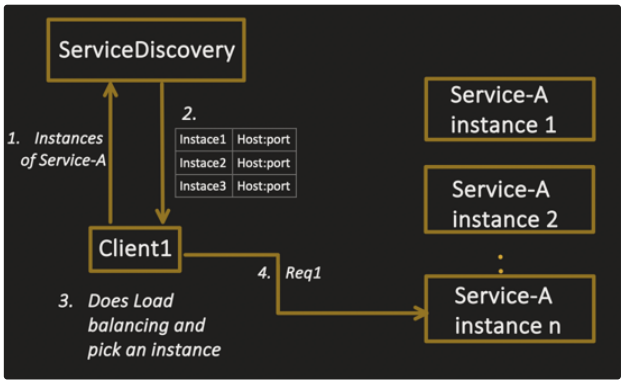
Server Side: Centralized Load Balancer like Nginx, ELB (Elastic Load balancer).
Like a separate microservice



Concept && Coding

Client Side: Load balancing capability present inside the client (the caller) or it uses a library to take the decision.

Like Spring cloud LoadBalancer, Netflix Ribbon, Istio with Sidecar etc.



<u>Client Side Load Balancer:</u>

<u>Pre-requisite :</u>

Let's see Client Side Load Balancing with RestTemplate:

In Service Discovery topic, we have seen that:

- After setting up the Service Discovery.
- In RestTemplate, we fetch the List of Instances using DiscoveryClient object.
- Then client is manually choosing the instances from the list and invoking the endpoint.

```java
@RestController
@RequestMapping("/orders")
public class OrderController {

    @Autowired
    DiscoveryClient discoveryClient;

    @Autowired
    RestTemplate restTemplate;

    @GetMapping("/{id}")
    public void callProductAPI(@PathVariable String id) {

        List<ServiceInstance> instances =
                discoveryClient.getInstances("product-service");

        URI uri = instances.get(0).getUri();

        String response = restTemplate.getForObject(
                uri + "/products/" + id,
                String.class
        );

        System.out.println("Response from Product api call is: " + response);
    }
}

```

Concept && Coding

```java
@Configuration
public class Config {

    @Bean
    public RestTemplate restTemplateObj() {
        return new RestTemplate();
    }
}
```

Now, lets use "Spring Cloud Load Balancer" and see the above example again:

1. Add dependency

```
<dependency>
<groupId>org.springframework.cloud</groupId>
<artifactId>spring-cloud-starter-loadbalancer</artifactId>
</dependency>


```

```
@RestController
@RequestMapping("/orders")
public class OrderController {

    @Autowired
    RestTemplate restTemplate;

    @Value("${product.service.baseurl}")
    String productBaseURL;

    @GetMapping("/{id}")
    public void callProductAPI(@PathVariable String id) {

        String response = restTemplate.getForObject(
                productBaseURL + "/products/" + id,
                String.class
        );

        System.out.println("Response from Product api call is: " + response);
    }
}

```

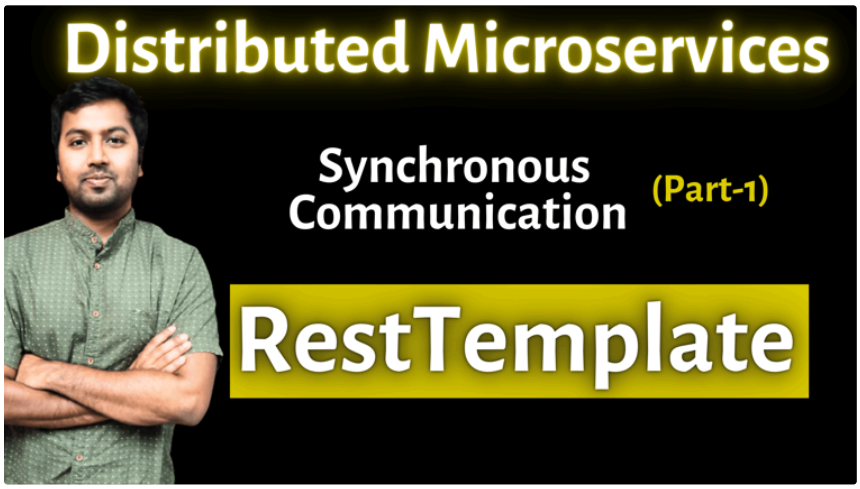
Concept && Coding

```
@Configuration
public class Config {

    @Bean
    /*
      With this annotation, framework start
      intercepting the request, call Service Discovery,
      get instances & apply load balancing algorithm.
    */
    @LoadBalanced
    public RestTemplate restTemplateObj() {
        return new RestTemplate();
    }
}

```

application.properties

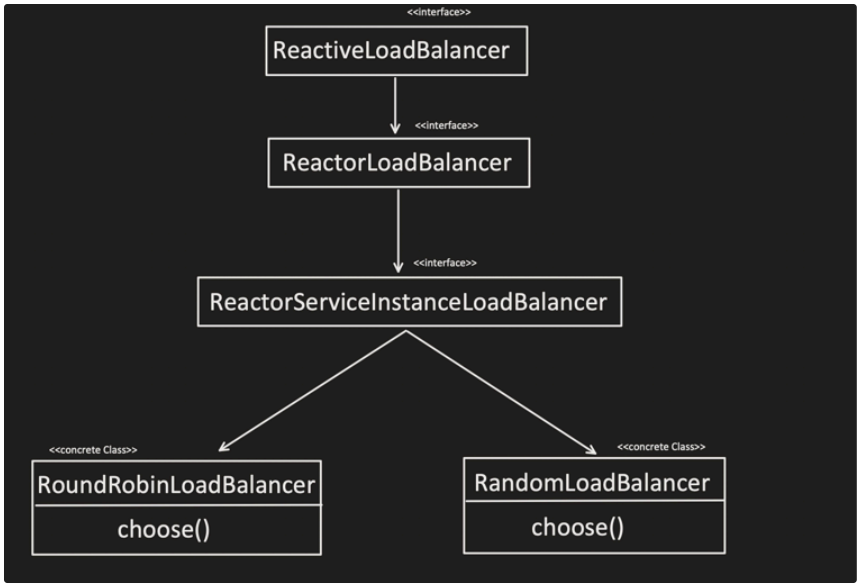
```
server.port=8081
spring.application.name=order-service

eureka.client.service-url.defaultZone=http://localhost:8761/eureka

product.service.baseurl=http://product-service


```

In RestTemplate topic, we have already seen below diagram and understood it in-depth:



Concept && Coding

- Each Load Balancing Algorithm is attached to a ServiceId. ServiceId is nothing but a client name like : "*product-service*".
- By default Spring Cloud Load Balancer picks RoundRobinLoadBalancer algorithm.
- Means if we are not going with Default Algorithm, then we need to define for which client (or serviceId) which algorithm we need.
- Spring Cloud Load Balancer supports only 2 algorithms i.e. RoundRobin and Random.
- If we want different algorithm like Weighted, Least Connection etc. then either we need to write custom implementation or switch to service mesh like Istio with Side car.



If we want to use say **RandomLoadBalancer** instead of Default **RoundRobinLoadBalancer**, we can override it in configuration

(Same as previous)

```
@RestController
@RequestMapping("/orders")
public class OrderController {

    @Autowired
    RestTemplate restTemplate;

    @Value("${product.service.baseurl}")
    String productBaseURL;

    @GetMapping("/{id}")
    public void callProductAPI(@PathVariable String id) {

        String response = restTemplate.getForObject(
                productBaseURL + "/products/" + id,
                String.class
        );

        System.out.println("Response from Product api call is: " + response);
    }
}

@Configuration
public class Config {

    @Bean
    @LoadBalanced
    public RestTemplate restTemplateObj() {
        return new RestTemplate();
    }
}

application.properties:
------------------------
server.port=8081
spring.application.name=order-service

```

Concept && Coding

```
eureka.client.service-url.defaultZone=http://localhost:8761/eureka

product.service.baseurl=http://product-service


```

Now Changing the Load Balancing algorithm for Product Service from default RoundRobinLoadBalancer to RandomLoadBalancer.

```
@SpringBootApplication
@EnableFeignClients
@LoadBalancerClient(
        name = "product-service",
        configuration = LoadBalancerProductClientConfig.class
)
public class OrderserviceApplication {

    public static void main(String[] args) {
        SpringApplication.run(OrderserviceApplication.class, args);
    }
}

```

*** Normal Config class(like above) generally invokes only at application startup.*

But this LoadBalancerClient config invokes at runtime too,

when "product-service" is used: "http://product-service/xyz"

Concept && Coding

```
@Configuration
public class LoadBalancerProductClientConfig {

    @Bean
    public ReactorLoadBalancer<ServiceInstance> productClientLoadBalancer(
            LoadBalancerClientFactory factory) {

        return new RandomLoadBalancer(
                factory.getLazyProvider(
                        "product-service",
                        ServiceInstanceListSupplier.class
                ),
                "product-service"
        );
    }
}

```

***ServiceInstanceListSupplier**, is actually the one, which helps to invoke Service Discovery and ask for instances of "product-service"*

New RandomLoadBalancer(*ServiceInstanceListSupplier, serviceId*)
we pass serviceId because it mapped this algorithm to this service ID only. So for "product-service" only RandomLoadBalancer is used.

What if, for "product-service" I want Algorithm A and for all other clients (or ServiceId), I want Algorithm B

```java
@SpringBootApplication
@EnableFeignClients

/*
   First child clients config will be
   loaded, then default one.
*/
@LoadBalancerClients(
        defaultConfiguration = LoadBalancerGlobalConfig.class,
        value = {
                @LoadBalancerClient(
                        name = "product-service",
                        configuration = LoadBalancerProductClientConfig.class
                )
        }
)
public class OrderserviceApplication {

    public static void main(String[] args) {
        SpringApplication.run(OrderserviceApplication.class, args);
    }
}

```

- As we already know, this *LoadBalancerClients* config is loaded at runtime for each service ID, So this *environment.getProperty(LoadBalancerClientFactory.PROPERTY_NAME)* will resolved at runtime.

- *@ConditionalOnMissingBean* is required at Global config class, because say for *"product-service"* two ReactorLoadBalancer bean should not get created one from *LoadBalancerProductClientConfig* and another from *LoadBalancerGlobalConfig*.

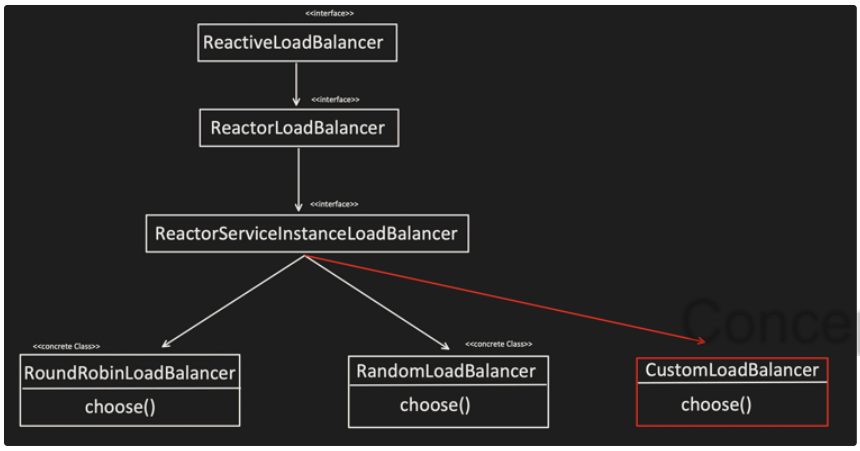
Concept && Coding

- So below, for "product-service" Random Load balancer algorithm will be used and for all other services "RoundRobin Algorithm will be used.

```java
@Configuration
@ConditionalOnMissingBean(ReactorLoadBalancer.class)
public class LoadBalancerGlobalConfig {

    @Bean
    public ReactorLoadBalancer<ServiceInstance> randomLoadBalancer(
            LoadBalancerClientFactory factory,
            Environment environment) {

        String serviceId = environment.getProperty(LoadBalancerClientFactory.PROPERTY_NAME);

        return new RoundRobinLoadBalancer(
                factory.getLazyProvider(
                        serviceId,
                        ServiceInstanceListSupplier.class
                ),
                serviceId
        );
    }
}

```

```java
@Configuration
public class LoadBalancerProductClientConfig {

    @Bean
    public ReactorLoadBalancer<ServiceInstance> productClientLoadBalancer(
            LoadBalancerClientFactory factory) {

        return new RandomLoadBalancer(
                factory.getLazyProvider(
                        "product-service",
                        ServiceInstanceListSupplier.class
                ),
                "product-service"
        );
    }
}

```

If we want to use say our **CustomLoadBalancer** logic:



Concept && Coding

```java
public class MyCustomLoadBalancer implements ReactorServiceInstanceLoadBalancer {

    private final ObjectProvider<ServiceInstanceListSupplier> serviceInstanceSuppliers;
    private final String serviceId;

    public MyCustomLoadBalancer(
            ObjectProvider<ServiceInstanceListSupplier> serviceInstanceSuppliers,
            String serviceId) {

        this.serviceInstanceSuppliers = serviceInstanceSuppliers;
        this.serviceId = serviceId;
    }

    @Override
    public Mono<Response<ServiceInstance>> choose(Request request) {

        return serviceInstanceSuppliers
                .getIfAvailable()
                .get()
                .next()
                .map(instances -> {
                    if (instances == null || instances.isEmpty()) {
                        return new EmptyResponse();
                    }
```

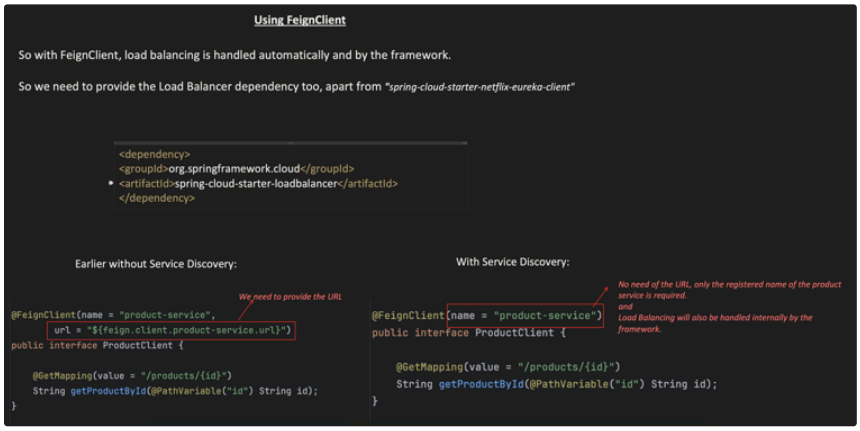
```
                    // Custom load balancing algorithm (demo: pick first instance)
                    return new DefaultResponse(instances.get(0));
                });
    }
}

```

```
@Configuration
public class LoadBalancerProductClientConfig {

    @Bean
    public ReactorLoadBalancer<ServiceInstance> productClientLoadBalancer(
            LoadBalancerClientFactory factory) {

        return new MyCustomLoadBalancer(
                factory.getLazyProvider(
                        "product-service",
                        ServiceInstanceListSupplier.class
                ),
                "product-service"
        );
    }
}

```

During Service Discovery, we have already seen how FeignClient handles load balancer automatically.

So for load balancer everything is same like RestTemplate, no change at all.

Concept && Coding




```
@RestController
@RequestMapping("/orders")
public class OrderController {

    @Autowired
    ProductClient productClient;

    @GetMapping("/{id}")
    public void callProductAPI(@PathVariable String id) {

        String response = productClient.getProductById(id);
        System.out.println("Response from Product api call is: " + response);
    }
}

```

```java
@SpringBootApplication
@EnableFeignClients
@LoadBalancerClient(
        name = "product-service",
        configuration = LoadBalancerProductClientConfig.class
)
public class OrderserviceApplication {

    public static void main(String[] args) {
        SpringApplication.run(OrderserviceApplication.class, args);
    }
}

```

```java
@Configuration
public class LoadBalancerProductClientConfig {

    @Bean
    public ReactorLoadBalancer<ServiceInstance> productClientLoadBalancer(
            LoadBalancerClientFactory factory) {

        return new RandomLoadBalancer(
                factory.getLazyProvider(
                        "product-service",
                        ServiceInstanceListSupplier.class
                ),
                "product-service"
        );
    }
}

```

Concept && Coding